

Here's a **comprehensive list of the functions/methods in the legacy `np.random` namespace** (i.e., what you get under `import numpy as np` and then `np.random. ....`). These are mostly *legacy* (older) functions — new code should use a **Generator** from `np.random.default_rng()` instead — but they're still present for backwards compatibility.

Legacy Random-Number Functions in `np.random`

Basic Random Numbers

These generate random numbers or arrays of floats/ints:

Float generators

- `np.random.random(size)` — random floats in `[0.0, 1.0)` (alias for `random_sample`)
- `np.random.random_sample(size)` — same as `random()`
- `np.random.ranf(size)` — another alias
- `np.random.rand(d0, d1, ...)` — uniform `[0,1)` array of shape `(d0, d1, ...)`

Integer generators

- `np.random.randint(low, high=None, size=None)` — random integers in `[low, high)`
- `np.random.random_integers(low, high, size)` — *deprecated* inclusive ints

Sampling from arrays

- `np.random.choice(a, size=None, replace=True, p=None)` — sample from 1-D array or range

Random bytes

- `np.random.bytes(length)` — random raw bytes

Permutations & Shuffling

- `np.random.shuffle(x)` — shuffle array *in place*
 - `np.random.permutation(x)` — return shuffled copy (or shuffled range)
-

Probability Distribution Samplers

These generate arrays of random values from statistical distributions (many are implemented internally but are exposed here):

- `np.random.beta(a, b, size)` — beta distribution
- `np.random.binomial(n, p, size)` — binomial distribution
- `np.random.chisquare(df, size)` — chi-square distribution
- `np.random.dirichlet(alpha, size)` — Dirichlet distribution
- `np.random.exponential(scale, size)` — exponential distribution
- `np.random.f(dfnum, dfden, size)` — F distribution
- `np.random.gamma(shape, scale, size)` — gamma distribution
- `np.random.geometric(p, size)` — geometric distribution
- `np.random.gumbel(loc, scale, size)` — Gumbel distribution
- `np.random.hypergeometric(ngood, nbad, nsample, size)` — hypergeometric
- `np.random.laplace(loc, scale, size)` — Laplace distribution
- `np.random.logistic(loc, scale, size)` — logistic distribution
- `np.random.lognormal(mean, sigma, size)` — log-normal distribution
- `np.random.logseries(p, size)` — log-series distribution
- `np.random.multinomial(n, pvals, size)` — multinomial
- `np.random.multivariate_normal(mean, cov, size)` — multivariate normal
- `np.random.negative_binomial(n, p, size)` — negative binomial
- `np.random.noncentral_chisquare(df, nonc, size)` — noncentral chi-square
- `np.random.noncentral_f(dfnum, dfden, nonc, size)` — noncentral F
- `np.random.normal(loc, scale, size)` — normal (Gaussian) distribution
- `np.random.pareto(a, size)` — Pareto distribution
- `np.random.poisson(lam, size)` — Poisson distribution
- `np.random.power(a, size)` — power distribution
- `np.random.rayleigh(scale, size)` — Rayleigh distribution
- `np.random.standard_cauchy(size)` — standard Cauchy
- `np.random.standard_exponential(size)` — standard exponential
- `np.random.standard_gamma(shape, size)` — standard gamma
- `np.random.standard_normal(size)` — standard normal ($\mu=0, \sigma=1$)
- `np.random.standard_t(df, size)` — standard Student's t
- `np.random.triangular(left, mode, right, size)` — triangular
- `np.random.uniform(low, high, size)` — uniform distribution
- `np.random.vonmises(mu, kappa, size)` — von Mises
- `np.random.wald(mean, scale, size)` — Wald distribution
- `np.random.weibull(a, size)` — Weibull distribution
- `np.random.zipf(a, size)` — Zipf distribution



Additional Utilities

There are also a few state-related and legacy functions accessible at `np.random`:

- `np.random.seed(seed)` — seed global RNG state (legacy)
 - `np.random.get_state()` / `np.random.set_state(state)` — get/set generator state
-

Modern Recommendation

Although all of those functions exist (and most remain documented), **new code should use a Generator** from the newer `numpy.random.default_rng()` API. That API groups random methods under the `rng` object, e.g.:

```
rng = np.random.default_rng(123)
rng.random((3,3))
rng.integers(0, 10, (5,))
rng.normal(0, 1, size=100)
```

This newer approach is:

- faster
- more statistically robust
- more consistent across versions